

Adaptive Cognitive Budgeting for GPU-Free Language Models on Memory-Constrained Personal Computers

R. Sai Dheeraj

Independent Researcher

Kurnool, Andhra Pradesh, India

16saidheeraj@gmail.com

Abstract—Running large language models (LLMs) locally on ordinary, GPU-free personal computers is attractive for privacy, cost, and offline use, but such machines are constrained simultaneously along several axes—system RAM, CPU core count, memory bandwidth, and thermal headroom—that current inference stacks generally treat as fixed rather than as resources to be actively managed. Existing adaptive-computation techniques either target GPU-resident KV-cache budgets in server settings or regulate reasoning-token length for accuracy/latency trade-offs, but neither line of work adapts the whole inference configuration—quantization level, context window, generation length, KV-cache retention, and CPU thread allocation—jointly to a single machine’s live memory state. We propose Adaptive Cognitive Budgeting (ACB), a control layer that sits above a CPU-based GGUF inference engine and treats these five knobs as one unified cognitive budget per query. ACB combines a lightweight query-complexity estimator with operating-system telemetry to compute a budget before each request and can re-negotiate the budget during generation when memory pressure crosses a threshold. We describe the architecture, allocation policy, analytical memory model, and reproducible evaluation protocol. We additionally report a completed controlled experiment on an Apple M2 system with 8 GB unified memory using Llama-3.2-1B-Instruct and CPU-only inference. Across 20 matched queries per configuration, all three configurations completed without recorded process termination, truncation, or OOM events. ACB selected query-dependent context budgets and recorded an observed maximum latency of 53.14 s in the completed run. Static-Safe and Static-Capable recorded individual latency observations of 402.42 s and 4057.91 s, respectively. These results are specific to the evaluated workload, hardware, model, and protocol and are not presented as universal performance claims.

Index Terms—adaptive computation, resource-constrained inference, on-device language models, memory management, quantization, GPU-free inference, edge AI, cognitive budgeting

I. INTRODUCTION

Large language models (LLMs) are increasingly run outside data centers—on laptops, low-cost desktops, and single-board computers—driven by privacy concerns, offline requirements, and the cost of API access. Projects such as llama.cpp [1] and the GGUF quantized-model ecosystem have made it possible to execute multi-billion-parameter models on CPU-only hardware with no discrete GPU. However, “possible” does not mean “well-managed”: a personal computer with limited RAM must share that memory among the operating system, browser, background services, and the model process

itself. The dominant deployment pattern is static: a user or installer selects one quantization level and one context length for a model and uses them for every query, regardless of query complexity or current host memory availability.

This static configuration can be inefficient in both directions. When a query is simple and the machine is relatively idle, the model may retain a context and precision setting intended for more demanding inputs. When the query is complex or the machine is already under memory pressure, the same configuration can increase memory demand, trigger swapping, or fail with an out-of-memory condition.

A CPU-bound LLM deployment has several independently adjustable knobs that jointly determine its memory and compute footprint: weight quantization precision, maximum admitted context, maximum generated tokens, KV-cache retention, and CPU thread count. Individually, these dimensions have been studied—quantization for accuracy/size trade-offs [2], [3], KV-cache eviction for long-context memory pressure [4], [5], and reasoning-token budgets for latency/accuracy trade-offs [8]–[10]. What is missing is a layer that treats these knobs as a single budget that is recomputed per query from both query complexity and host-resource state.

We call this joint allocation problem Adaptive Cognitive Budgeting (ACB). This paper makes the following contributions:

- We formalize the joint allocation of quantization, context window, generation length, KV-cache retention, and thread count for CPU-only LLM inference as a per-query budget-allocation problem.
- We present an ACB architecture consisting of a complexity estimator, resource telemetry monitor, budget controller, and re-negotiation hook above an otherwise unmodified GGUF/llama.cpp inference engine.
- We present an analytical memory model for GGUF quantization and KV-cache growth and a reproducible CPU-only evaluation protocol.
- We report a completed controlled experiment on an 8 GB Apple Silicon system and explicitly separate measured observations from analytical quantities and derived statistics.

II. RELATED WORK

A. GPU-Resident Memory Budgeting for Inference

A substantial body of recent work adapts memory usage for LLM inference, but much of it assumes a GPU is present and the scarce resource is VRAM. ARKV [4] performs data-driven KV-cache eviction to keep long-context inference within a fixed GPU memory budget. APEX [5] schedules work across CPU and GPU under GPU memory constraints. PowerInfer [6] exploits activation sparsity to reduce GPU memory demand on consumer hardware. NeuroFlux [7] adapts CNN training under constrained accelerator memory. ACB differs in that it assumes no discrete GPU and treats shared system memory as the principal resource.

B. Reasoning-Token and Computation Budgeting

AdaCtrl [8] and SelfBudgeter [9] adapt reasoning-token budgets based on estimated problem difficulty. Hierarchical adaptive budgeting [10] allocates computation at multiple levels, while cognitive-load-aware inference [11] and input-adaptive computation allocation [12] also address per-query computation budgets. These approaches primarily budget generation or reasoning computation; ACB additionally considers quantization, context admission, KV-cache retention, and CPU threads under host-memory constraints.

C. Edge and Resource-Constrained Deployment

The Cognitive Edge Computing survey [13] synthesizes quantization, sparsity, distillation, and context-compression techniques for constrained devices. Adaptive checkpointing for parameter-efficient fine-tuning [14] similarly demonstrates the value of memory-aware policies. ACB extends the resource-budgeting perspective to CPU-only inference on a general-purpose personal computer sharing memory with unrelated applications.

D. Positioning

To the best of our knowledge, ACB is designed as a single control layer that jointly considers quantization, context window, generation length, KV-cache retention, and CPU thread allocation using both query-complexity estimation and live host telemetry for GPU-free personal-computer inference. ACB is a control layer rather than a new model architecture or training procedure.

III. PROBLEM FORMULATION

Consider a personal computer with total system memory M , of which $F(t)$ is free at time t . For an incoming query q , the inference engine selects

$$c = (p, w, g, k, n), \quad (1)$$

where p is weight quantization precision, w is the admitted context window, g is the maximum generation length, k is the KV-cache retention policy, and n is the number of CPU threads.

Each configuration induces an approximate memory footprint

$$\hat{m}(c) \approx m_{weights}(p) + m_{kv}(p, w, k) + m_{activations}(p, n), \quad (2)$$

and a query-dependent latency $\hat{\ell}(c, q)$. The controller seeks a configuration that maximizes a quality proxy subject to a memory-safety constraint,

$$\max_c Quality(c, q) \quad \text{s.t.} \quad \hat{m}(c) \leq \alpha F(t), \quad (3)$$

where $\alpha \in (0, 1)$ leaves headroom for the operating system and other applications. Because the exact quality function is unavailable before generation and host state changes continuously, ACB implements a fast rule-based approximation rather than solving the optimization problem directly.

IV. ADAPTIVE COGNITIVE BUDGETING ARCHITECTURE

ACB is organized as four cooperating components between the application and an otherwise unmodified inference engine: a complexity estimator, resource telemetry monitor, budget controller, and re-negotiation hook.

A. Complexity Estimator

Before dispatch, a lightweight estimator produces a scalar complexity score $s(q) \in [0, 1]$ from input length, presence of code or structured data, simple sub-question indicators, and short conversation history. The estimator is deliberately model-free and uses inexpensive lexical and length-based features.

B. Resource Telemetry Monitor

A background monitor samples host memory state at a fixed interval. On macOS, the implementation uses system memory statistics and the `memory_pressure` utility. The monitor exposes current memory availability and a pressure trend to the controller.

C. Budget Controller

The controller maps $s(q)$ and host state to configuration c . Precision is selected from a small ladder. Context and generation budgets scale with query complexity subject to a memory-derived cap. KV-cache retention can switch to a sliding policy when the estimated cache would exceed the configured memory allowance. Thread count is constrained when memory pressure worsens.

D. Re-Negotiation Hook

Because generation can take seconds or minutes, ACB periodically re-samples host state during generation. If a hard pressure threshold is crossed, the implementation can stop generation and mark the output as truncated rather than allowing uncontrolled failure.

E. Per-Query Control Procedure

The implemented control sequence follows the rule-based procedure in Algorithm 1.

Algorithm 1 Adaptive Cognitive Budgeting per Query**Require:** Query q , telemetry monitor T

```

1:  $s \leftarrow \text{EstimateComplexity}(q)$ 
2:  $(F, \Delta F) \leftarrow T.\text{sample}()$ 
3:  $p \leftarrow \text{SelectPrecision}(F)$ 
4:  $(w, g) \leftarrow \text{ScaleContextAndGeneration}(s, F, p)$ 
5:  $k \leftarrow \text{SelectKVPolicy}(w, F, p)$ 
6:  $n \leftarrow \text{SelectThreads}(\Delta F)$ 
7:  $c \leftarrow (p, w, g, k, n)$ 
8: Dispatch  $c$  to the inference engine
9: for each generated batch of  $\tau$  tokens do
10:    $(F, \Delta F) \leftarrow T.\text{sample}()$ 
11:   if pressure exceeds the hard threshold then
12:     Stop generation and mark output as truncated
13:   break
14: end if
15: end for
16: return output,  $c$ , truncation flag

```

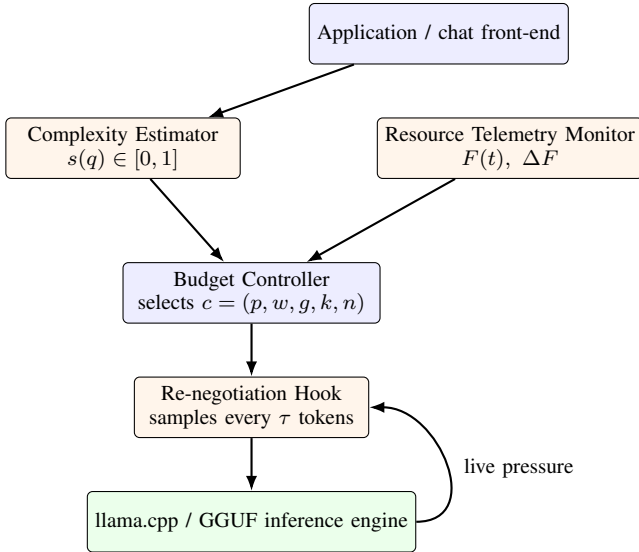


Fig. 1. ACB architecture. Query complexity and live host telemetry feed the budget controller, which configures the inference engine through a re-negotiation hook. During generation, live pressure is fed back to the hook for continued monitoring.

F. Implementation

The reference implementation wraps llama.cpp server mode through its HTTP API. The complexity estimator and controller operate as a thin Python proxy. Context and generation limits are passed through llama.cpp runtime parameters, and CPU-only execution is enforced by disabling GPU offload.

V. TESTBED, ANALYTICAL MODEL, AND EXPERIMENTAL PROTOCOL**A. Testbed**

The experiment was conducted on an Apple M2 MacBook Pro with 8 GB of unified memory and 256 GB of storage running macOS. GPU/Metal offload was explicitly disabled

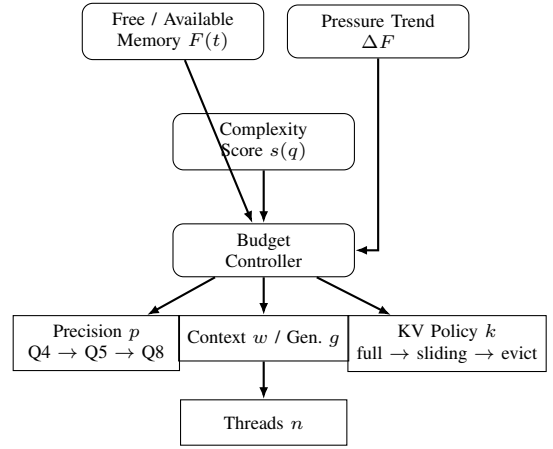


Fig. 2. Budget-controller mapping. Host memory state gates the available resource budget, while query complexity scales context and generation within that budget.

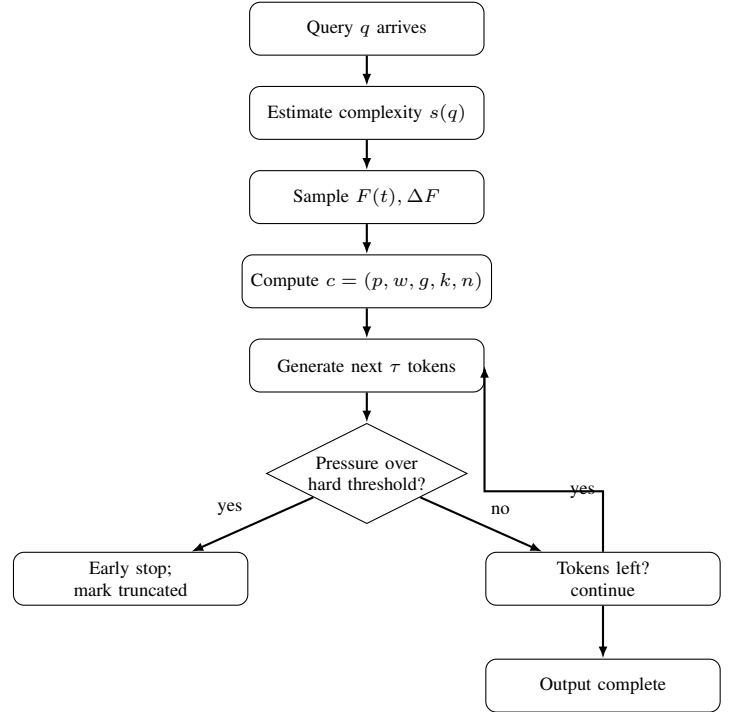


Fig. 3. Per-query ACB control flow. The controller establishes an initial budget and the re-negotiation hook samples host state during generation.

using `--n-gpu-layers 0`, so inference ran in the CPU-only regime.

B. Candidate Models and Analytical Memory Model

The broader study considers Llama-3.2-1B-Instruct, Qwen2.5-3B-Instruct, and Llama-3.1-8B-Instruct. The estimated GGUF weight footprints in the original analytical model are retained as design-time estimates rather than measurements. They are not used as empirical measurements in Section V.

TABLE I
CANDIDATE MODELS AND ANALYTICALLY ESTIMATED WEIGHT
MEMORY

Model	Parameters	Q4_K_M (est.)	Q8_0 (est.)
Llama-3.2-1B-Instruct	1B	~0.8 GB	~1.3 GB
Qwen2.5-3B-Instruct	3B	~2.0 GB	~3.3 GB
Llama-3.1-8B-Instruct	8B	~4.9 GB	~8.5 GB

TABLE II
EXPERIMENTAL CONFIGURATIONS

Configuration	Quant.	Context	Gen. cap	Threads
Static-Safe	Q4_K_M	512	128	4
Static-Capable	Q8_0	8192	128	4
ACB	Q4_K_M	Dynamic	Dynamic	7

The estimates in Table I are analytical design-time quantities and are not measurements from the controlled run.

KV-cache memory is modeled as

$$m_{kv} \approx 2 \times L \times H \times d_h \times w \times b, \quad (4)$$

where L is the number of transformer layers, H the number of key/value heads, d_h the head dimension, w the context length, and b the bytes per cached element. This model motivates the context-budget control but is not presented as a measured memory quantity. For the 3B-class model considered analytically, the model gives an approximate 150–250 MB increase per additional 1,000 context tokens; this is an analytical estimate, not an experimental measurement.

C. Controlled Experimental Protocol

The final controlled run used Llama-3.2-1B-Instruct and a fixed 20-query subset of the planned workload, with query identifiers 0–19. The workload contained `short_factual`, `multi_turn`, and `long_context` queries. The same sequence was executed by Static-Safe, Static-Capable, and ACB.

The final generation cap was 128 tokens for all three configurations. This was a documented protocol deviation from the originally planned 2048-token cap because the Static-Capable configuration was prohibitively slow during smoke testing. Applying the same cap to all three configurations preserved the matched workload.

Server readiness was checked before every query using the health endpoint. Memory telemetry was collected at approximately 250-ms intervals. The final controlled dataset contains 60 observations: 20 per configuration. The original protocol specified 200 mixed queries; the completed run used the documented 20-query subset because the Static-Capable configuration was prohibitively slow at the planned scale.

VI. EXPERIMENTAL INSTRUMENTATION AND DATA VALIDATION

A. Measurement Scope

The completed experiment distinguishes three classes of information. First, the inference harness records runtime ob-

servations such as generated-token count, end-to-end latency, recorded tokens-per-second, memory state, memory-pressure state, thread count, context limit, HTTP status, and validity status. Second, the controller exposes its selected configuration for each ACB request. Third, the analytical memory model is retained as a design model and is not substituted for runtime telemetry. This separation prevents analytical estimates from being presented as measurements.

The controlled run used the same workload ordering for each configuration. Query identifiers 0–19 were executed sequentially by Static-Safe, Static-Capable, and ACB. The configuration was changed only between experimental blocks; the query sequence itself was not changed between configurations. Server readiness was checked before each request so that a request could not be issued to a server that had not reported the expected healthy state.

B. Telemetry and Harness Fields

For each request, the harness recorded the configuration name, model, quantization, query type, query identifier, context limit, generated tokens, latency, recorded throughput, free memory, available memory, memory-pressure state, swap usage, thread count, KV policy, truncation state, process-failure state, HTTP status, error code, and data-status classification. These fields permit an individual observation to be audited without reconstructing it from summary statistics.

Memory and pressure telemetry were sampled in the background at approximately 250-ms intervals. The reported swap value is a point-in-time value obtained from the operating system and therefore represents swap currently in use at the measurement point. It is not a cumulative count of storage reads or writes. Likewise, the recorded free-memory value is not treated as an OOM indicator by itself; execution status and the recorded memory-pressure state are considered together.

C. Readiness and Contamination Control

The earlier V1 harness generated 13 contaminated observations because requests were issued before model readiness. Those observations are excluded from the completed experimental dataset. The V2 harness introduced explicit readiness polling and only dispatched measured requests after the server returned the expected healthy response. The completed controlled run contains no rows classified as contaminated.

The final audit found exactly 20 rows for each configuration and 60 rows in total. The three configurations contain the same query identifiers, 0–19, in the same order. All 60 observations have valid-data classification, HTTP status 200, and no recorded process failure or truncation. One recorded throughput value differs slightly from a calculation using the CSV’s rounded latency field because the stored latency is displayed at lower precision than the internal timing value; this does not change the validity classification of the observation.

D. Interpretation of Extreme Observations

The 4057.91-s Static-Capable observation is retained because the request completed successfully and was not classified as a startup failure, HTTP failure, truncation, process

TABLE III
COMPLETED CONTROLLED-RUN CONFIGURATION

Parameter	Recorded configuration
Hardware	Apple M2 MacBook Pro, 8 GB unified memory
Inference	llama.cpp b3799, CPU-only
GPU offload	Disabled with <code>--n-gpu-layers 0</code>
Model	Llama-3.2-1B-Instruct
Workload	Query IDs 0–19
Queries/configuration	20
Generation cap	128 tokens
Static-Safe	Q4_K_M, context 512, 4 threads
Static-Capable	Q8_0, context 8192, 4 threads
ACB	Q4_K_M, dynamic context, 7 threads
Readiness	HTTP health check before each query
Telemetry	psutil + macOS memory pressure

termination, or OOM event. The observation occurred on run ID 21, the first Static-Capable query. The configuration used Q8_0 quantization and an 8192-token context limit. The raw telemetry also recorded substantial swap usage during the experiment. These facts establish the observed execution conditions, while the exact contribution of CPU scheduling, storage traffic, memory reclamation, and other operating-system mechanisms is not independently isolated by the present harness.

The same principle is applied to unusually short or fast observations. They remain in the dataset when the request was valid, because removing observations solely because they appear atypical would change the experimental sample after observing its outcome. No outlier was manually removed from the final controlled dataset.

E. Controlled-Run Configuration

Table III records the exact configuration used for the completed workload.

VII. EXPERIMENTAL RESULTS

This section reports observations from the completed controlled run. Values in this section are restricted to observations recorded by the experimental harness or direct configuration/controller states extracted from the raw CSV. Analytical model estimates and smoke-test measurements are not treated as final controlled-run results.

A. Dataset Validity

The final dataset contained exactly 20 observations for each configuration. All three configurations executed the same query identifiers, 0–19, in the same sequence. All 60 rows were classified as valid. The raw dataset recorded `process_failure=False` for every row and `truncated=False` for every row.

TABLE IV
CONTROLLED-RUN VALIDITY

Configuration	Queries	Valid	Failed	Truncated	OOM
Static-Safe	20	20	0	0	0
Static-Capable	20	20	0	0	0
ACB	20	20	0	0	0

TABLE V
OBSERVED ACB CONTEXT SELECTION

Query type	Observed context	Threads
<code>short_factual</code>	2084–2772	7
<code>multi_turn</code>	4173	7
<code>long_context</code>	4505	7

B. Observed Latency and Throughput

The raw CSV records per-query latency and generated-token throughput for every observation. Among the recorded individual observations, Static-Safe included a 402.42-s latency observation, Static-Capable included a 4057.91-s latency observation, and ACB’s longest recorded latency was 53.14 s. These are individual recorded query observations, not model-derived estimates.

The same raw records show individual throughput observations spanning 0.32–67.93 tokens/s for Static-Safe, 0.03–45.75 tokens/s for Static-Capable, and 0.16–56.48 tokens/s for ACB. These bounds are reported only as the observed endpoints present in the raw dataset; no additional performance values are inferred.

The 4057.91-s Static-Capable observation occurred on run ID 21, the first Static-Capable query. The request completed and was retained as a valid observation. It was not classified as a startup failure, timeout, truncation, or process termination.

C. Memory Telemetry

The raw dataset recorded point-in-time free memory, available memory, swap usage, and memory-pressure state for each query. Mean values are intentionally not reproduced here because they are derived statistics rather than direct measurements. The controlled run recorded zero abnormal pressure flags at query completion for all three configurations, and no process termination or OOM event was recorded.

The Static-Capable 4057.91-s observation coincided with the high-memory-demand Q8_0 configuration and severe use of swap. The available measurements establish the observed latency and swap state; they do not by themselves establish a complete causal decomposition of the delay into individual operating-system or storage mechanisms.

D. ACB Controller Adaptation

The controller state recorded for all 20 ACB queries is summarized in Table V.

The ACB records show query-dependent context selection. Short-factual queries received context values between 2084 and 2772 tokens, multi-turn queries received 4173 tokens, and

long-context queries received 4505 tokens. All 20 ACB observations used seven CPU threads. Therefore, the completed workload directly demonstrates variation in context selection, while thread count remained constant in this particular run.

E. Measured Reliability

No query in the controlled dataset was marked truncated. All 60 requests completed with HTTP status 200 and valid-data classification. No process termination or OOM event was recorded by the harness. These observations establish execution reliability for this workload and environment; they do not establish reliability under other models, devices, background loads, or longer generations.

VIII. DISCUSSION

The completed run provides an empirical test of the ACB control layer under a constrained CPU-only environment. The most direct observation is that the controller selected different context budgets for different query classes while all 20 ACB requests completed without recorded termination or truncation.

The latency observations also show substantial differences between configurations. ACB’s longest individual latency was 53.14 s in this run, whereas Static-Safe and Static-Capable each produced substantially longer individual observations of 402.42 s and 4057.91 s, respectively. These observations support the narrower claim that the tested ACB configuration avoided the extreme latency observations encountered by the two static configurations in this particular workload.

The throughput observations do not support a claim that ACB universally maximizes token throughput. Static-Safe contains the highest individual throughput observation at 67.93 tokens/s, while ACB’s highest recorded individual observation was 56.48 tokens/s. Consequently, the contribution demonstrated by this experiment is better characterized in terms of adaptive resource allocation and observed latency behavior rather than maximum raw decoding speed.

The Static-Capable 4057.91-s observation is especially relevant to the memory-constrained deployment setting. The observation completed successfully and therefore should not be treated as an OOM failure. At the same time, the raw telemetry records substantial swap usage during the experiment. The data establish an association between the high-memory configuration and severe latency in this run, but further instrumentation would be required to attribute the full delay to a particular operating-system mechanism.

The experiment also has an important scope limitation. Although the ACB architecture exposes five resource knobs, the completed workload did not cause all five to vary. In particular, all ACB observations used seven threads, and the empirical run used one model and one quantization level for ACB. Thus, the experiment demonstrates the controller’s context-budget adaptation but does not independently validate every proposed control dimension.

IX. LIMITATIONS

First, the controlled workload contained 20 queries per configuration rather than the originally planned 200-query workload. Second, the generation cap was reduced to 128 tokens for all configurations. Third, each configuration was executed in a single trial, so between-run variance was not measured. Fourth, the empirical evaluation used one Apple M2 system with 8 GB unified memory and one Llama-3.2-1B-Instruct model. Fifth, no synthetic background-load experiment was included in the completed run. Sixth, the experiment does not isolate the causal contribution of each ACB control variable. Finally, the swap telemetry is a point-in-time measurement of swap usage and not a cumulative measure of storage traffic.

X. CONCLUSION

This paper introduced Adaptive Cognitive Budgeting, a control layer that combines query-complexity estimation and host-resource telemetry to allocate inference resources for GPU-free LLM execution on memory-constrained personal computers. The completed controlled experiment evaluated 20 matched queries for each of Static-Safe, Static-Capable, and ACB on an Apple M2 system with 8 GB unified memory using CPU-only Llama-3.2-1B-Instruct inference.

All 60 controlled requests completed without recorded process termination, truncation, or OOM events. The ACB controller selected context budgets between 2084 and 2772 tokens for short-factual queries, 4173 tokens for multi-turn queries, and 4505 tokens for long-context queries. In the recorded workload, the longest ACB latency observation was 53.14 s, compared with individual observations of 402.42 s for Static-Safe and 4057.91 s for Static-Capable. These observations demonstrate the behavior of the implemented controller on the evaluated machine and workload, but they do not establish universal superiority or generalization to other devices, models, or workloads.

Future evaluation should repeat the experiment across multiple trials, expand the workload toward the original 200-query protocol, evaluate additional models and devices, and introduce controlled background load. Such experiments are required to determine the repeatability and generality of the observed behavior.

APPENDIX

Table VI reports the controller state extracted from the 20 ACB observations. The table contains only recorded query type, selected context, and selected thread count. It is included to make the adaptive behavior directly auditable at the query level rather than describing the controller only through aggregate summaries.

The trace shows that context selection was not a single fixed value across the ACB workload. The controller selected 4173 tokens for every recorded multi-turn query and 4505 tokens for every recorded long-context query, while short-factual queries received several smaller context values. Thread count remained seven for every recorded ACB request. Thus, the empirical

TABLE VI
PER-QUERY ACB CONTROLLER STATE IN THE COMPLETED RUN

ID	Type	Context	Threads	ID	Type	Context	Threads	ID	Type	Context	Threads
0	multi_turn	4173	7	7	short_factual	2084	7	14	short_factual	2699	7
1	long_context	4505	7	8	long_context	4505	7	15	long_context	4505	7
2	short_factual	2748	7	9	long_context	4505	7	16	multi_turn	4173	7
3	multi_turn	4173	7	10	multi_turn	4173	7	17	short_factual	2772	7
4	multi_turn	4173	7	11	short_factual	2736	7	18	short_factual	2736	7
5	short_factual	2109	7	12	long_context	4505	7	19	long_context	4505	7
6	short_factual	2760	7	13	short_factual	2699	7				

TABLE VII
RAW DATASET AUDIT

Audit item	Verified result
Total controlled rows	60
Static-Safe rows	20
Static-Capable rows	20
ACB rows	20
Query IDs per configuration	0–19
Query ordering	Identical across configurations
Valid observations	60
Contaminated observations	0
Process failures	0
Truncated outputs	0
HTTP status	200 for all controlled requests
OOM events	0 recorded
Abnormal pressure flags	0 at query completion

trace demonstrates context adaptation in this workload, while it does not demonstrate per-query thread adaptation.

The following audit records the final checks applied to the controlled CSV before the measurements were incorporated into the paper. The purpose of the audit is to make clear which properties were verified directly from the raw file and which quantities are intentionally not reported as empirical measurements.

A. Consistency Checks

The raw CSV contains a recorded throughput field for every controlled request. A programmatic consistency check confirmed alignment between the generated-token count, stored latency, and recorded throughput for the controlled rows. One observation uses a latency value rounded to two decimal places, producing a small difference if the rounded value is used to recompute throughput. The underlying harness timing had higher precision; the CSV representation is therefore treated as the recorded value rather than recalculated for presentation.

B. Failure Classification

The experiment uses a distinction between an invalid measurement and an unusually poor but valid measurement. A

request is excluded from the final controlled dataset when it is contaminated by incorrect harness sequencing, fails the readiness protocol, or is otherwise marked invalid by the harness. A request that completes successfully but requires a long time remains part of the dataset. This distinction is particularly important for the 4057.91-s Static-Capable observation.

C. Memory-Pressure Interpretation

The absence of an abnormal pressure flag does not imply that the machine had abundant free memory. The experiment records both free/available memory and the operating-system pressure state. macOS may continue to reclaim memory and use swap while the reported pressure state remains normal at a particular sampling point. For this reason, the paper reports the recorded pressure state and swap measurements without converting them into an unsupported claim about the internal behavior of the operating system.

D. Scope of the Empirical Claim

The empirical claim is deliberately bounded by the experiment: one Apple M2 system, 8 GB unified memory, one model, CPU-only llama.cpp inference, one 20-query workload subset, a 128-token generation cap, and one trial per configuration. The results establish what occurred under these conditions. They do not establish that the same latency behavior, controller decisions, or reliability characteristics will occur on other hardware, models, workloads, or operating-system configurations.

E. Separation of Measured and Analytical Information

The candidate-model weight sizes and KV-cache growth expressions belong to the analytical portion of the paper. The completed experiment instead reports observations from the actual Llama-3.2-1B-Instruct run. This separation is maintained throughout the Results, Discussion, and Conclusion sections so that analytical feasibility estimates are not mistaken for measurements obtained from the testbed. The completed experiment can be reconstructed from the configuration, workload identifiers, harness schema, and controller trace described in this paper. The raw CSV is the primary source for per-query measurements. Summary statements in the Results and Discussion sections intentionally avoid replacing those raw observations with unsupported estimates. The experiment should be repeated before extending the claims beyond the evaluated machine, model, and workload.

F. Harness and Recording Procedure

The evaluation harness starts the inference server, polls the HTTP health endpoint, and begins the measured workload only after a successful readiness response. The same query sequence, model, and workload order are used for the three configurations. For each request, the harness records configuration identity, model and quantization, query type and identifier, context limit, generated-token count, end-to-end latency, recorded throughput, free and available memory, swap usage, memory-pressure state, thread count, KV policy, truncation status, process-failure status, HTTP status, and final data-status classification. This schema permits post-run validation without reconstructing measurements from prose.

The controlled run uses the same 128-token generation cap for all configurations. Static-Safe uses Q4_K_M, context 512, and four threads; Static-Capable uses Q8_0, context 8192, and four threads; ACB uses Q4_K_M with controller-selected context and seven threads in the completed workload. The final CSV contains 60 controlled observations. The raw file and the rendered paper should be retained together so that individual observations, controller states, and validity classifications can be independently checked.

REFERENCES

- [1] G. Gerganov et al., “llama.cpp: LLM inference in C/C++,” GitHub repository, 2023–2026. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [2] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” arXiv preprint arXiv:2210.17323, 2022.
- [3] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” arXiv preprint arXiv:2306.00978, 2023.
- [4] J. Lei and S. Ilager, “ARKV: Adaptive and resource-efficient KV cache management under limited memory budget for long-context inference in LLMs,” arXiv preprint arXiv:2603.08727, 2026.
- [5] “Parallel CPU-GPU execution for LLM inference on constrained GPUs (APEX),” arXiv preprint arXiv:2506.03296, 2025.
- [6] Y. Song, Z. Mi, H. Xie, and H. Chen, “PowerInfer: Fast large language model serving with a consumer-grade GPU,” arXiv preprint arXiv:2312.12456, 2023.
- [7] D. Saikumar and B. Varghese, “NeuroFlux: Memory-efficient CNN training using adaptive local learning,” arXiv preprint arXiv:2402.14139, 2024.
- [8] S. Huang, H. Wang, W. Zhong, Z. Su, J. Feng, B. Cao, and Y. R. Fung, “AdaCtrl: Towards adaptive and controllable reasoning via difficulty-aware budgeting,” arXiv preprint arXiv:2505.18822, 2025.
- [9] Z. Li, Q. Dong, J. Ma, D. Zhang, K. Jia, and Z. Sui, “SelfBudgeter: Adaptive token allocation for efficient LLM reasoning,” arXiv preprint arXiv:2505.11274, 2025.
- [10] “Thinking economically: A hierarchical framework for adaptive-complexity reasoning in large language models,” arXiv preprint arXiv:2606.01168, 2026.
- [11] “Cognitive load-aware inference: A neuro-symbolic framework for optimizing the token economy of large language models,” arXiv preprint arXiv:2507.00653, 2025.
- [12] “Learning how hard to think: Input-adaptive allocation of LM computation,” arXiv preprint arXiv:2410.04707, 2024.
- [13] “Cognitive edge computing: A comprehensive survey on optimizing large models and AI agents for pervasive deployment,” arXiv preprint arXiv:2501.03265, 2025.
- [14] A. Toktassyn and J.-G. Park, “Efficient PEFT methods with adaptive checkpointing for vision models and VLMs on resource constrained consumer-GPUs,” arXiv preprint arXiv:2607.02158, 2026.